

MOUNT KENYA UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY



COURSE CODE: BIT3208

COURSE TITLE: ADVANCED WEB DESIGN AND DEVELOPMENT

PROJECT TITLE: E-COMMERCE WEBSITE DEVELOPMENT

BY:

NAME: BON LIVINGSTON

REGISTRATION NUMBER: BBIT/2024/51875

SUBMITTED TO:

LECTURER: Mr. NYORO

DATE OF SUBMISSION: JUNE 2026

BIT3208: ADVANCED WEB DESIGN AND DEVELOPMENT

PROJECT TITLE: E-Commerce Website Development Using PHP and MySQL

SELECTED TECHNOLOGIES:

- HTML
- CSS
- JavaScript
- PHP
- MySQL
- XAMPP
- Visual Studio Code

Online portfolio/ Github link;

<https://github.com/bon-livingston/BIT3208-Advance-web-project>

IMPORTANCE

Version Control: Enables secure tracking and deployment management of web scripts.

Professional Documentation: Provides centralized code repository hosting for clear system architecture review.

Portfolio Building & Industry readiness: Showcases real world full-stack database application skills.

TABLE OF CONTENT

WEEK 1: Local Environment setup

Installation of XAMPP.....	3
Apache and MySQL running.....	4
Local host test page.....	4
Hello world test.....	5.
Database connection test.....	6
My reflection.....	7

WEEK 2: WIREFRAME AND GUI DESIGN

Dashboard layout design.....	8
Mobile view mockup.....	9
Color theme and navigation structure.....	9
GUI prototype in Fig ma.....	10
My reflection.....	11

Week 3: JAVA SCRIPT AND PHP BASICS

JavaScript form validation.....	12
Password strength checker or input validation.....	12
PHP syntax practice.....	13
Database connection script.....	13
Dynamic user input handling.....	13

WEEK 4: SERVER SIDE COMPONENTS AND BACKEND DEVELOPMENTFOUNDATIONS

Understanding server side programming.....	14
HTML form.....	15

Simple authentication system.....	15
Backend Organization.....	16
Reflection.....	16

WEEK 5: ADVANCED WEB DESIGN AND DEVELOPMENT

Introduction to database.....	17
Database creation.....	18
Creating tables.....	18
CRUD operation.....	19
Reflection.....	21

=====

WEEK 1: LOCAL ENVIRONMENT SETUP

Title:

Installation and Testing of Local Development Environment

Required Evidence

Fig 1: Installation of XAMPP

Caption:

Successful installation of XAMPP development environment on Windows operating system.

[June 1, 2026]

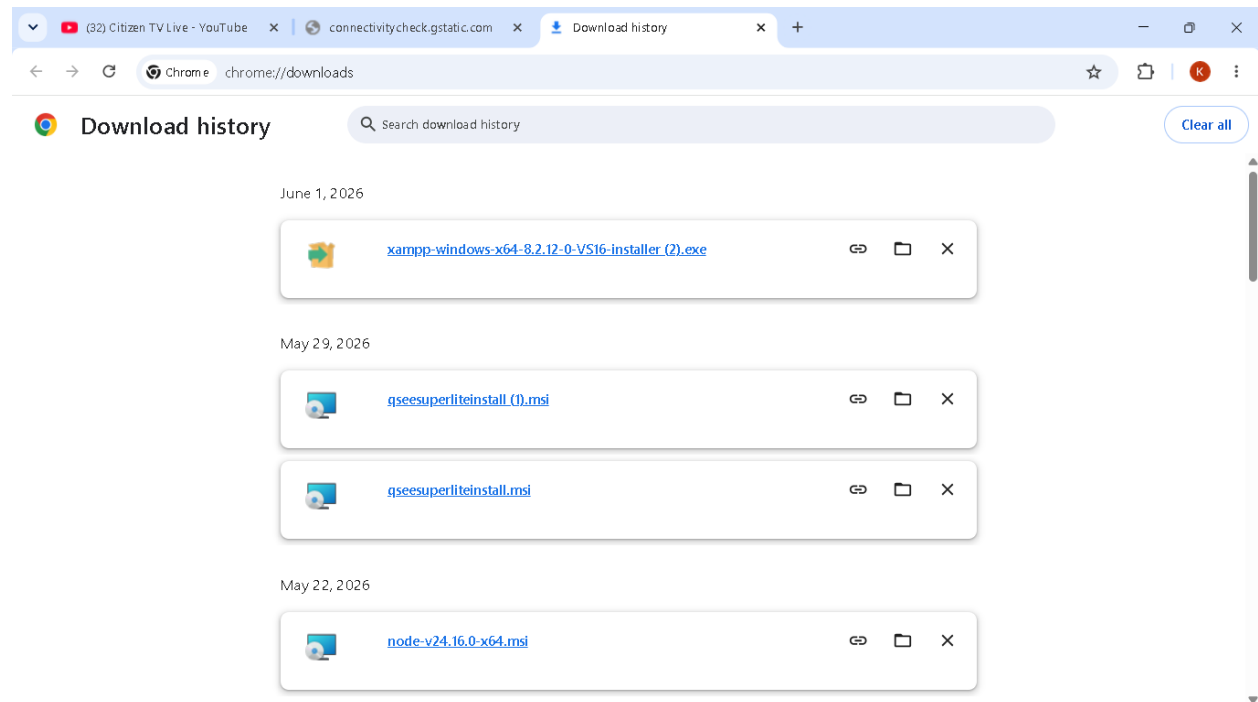


Fig 2: Apache and MySQL Running

Caption:

Apache and MySQL services successfully started using the XAMPP Control Panel.

[Insert Screenshot]

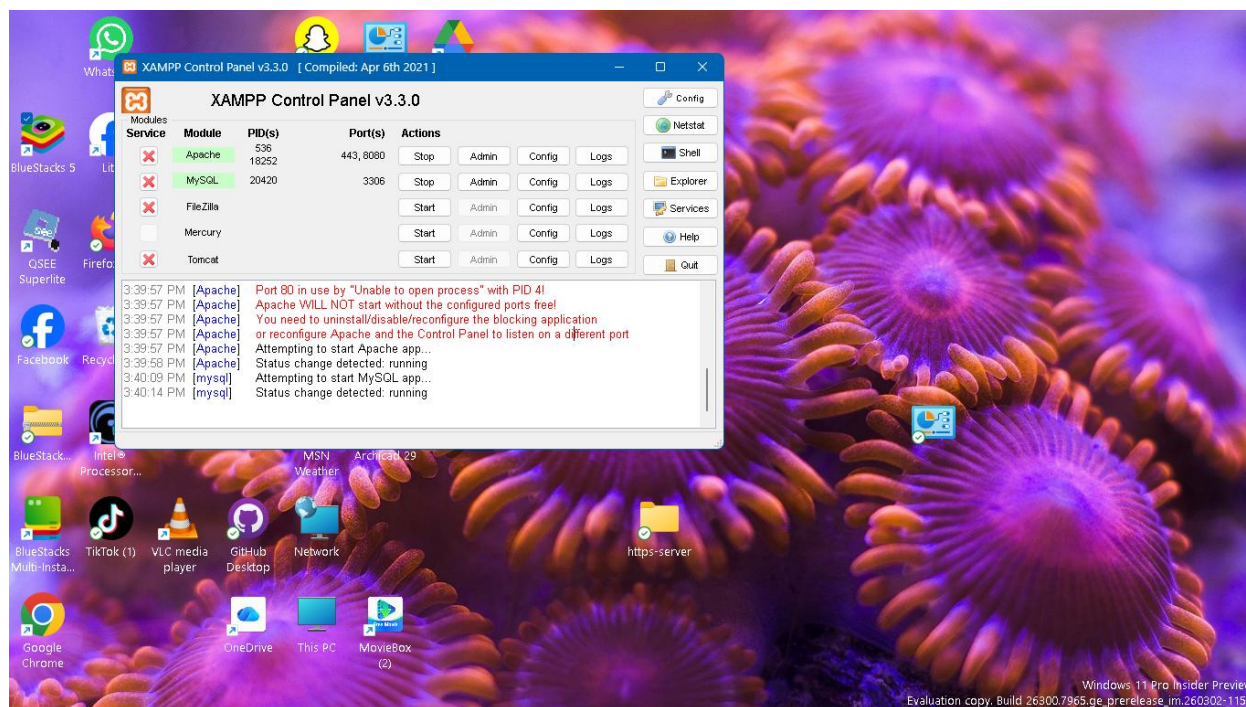


Fig 3: Local host Test Page

Caption:

Browser successfully displaying local host page confirming Apache server is working.



Fig 4: Hello World Test

Caption:

First PHP page executed successfully showing "Hello World" output.

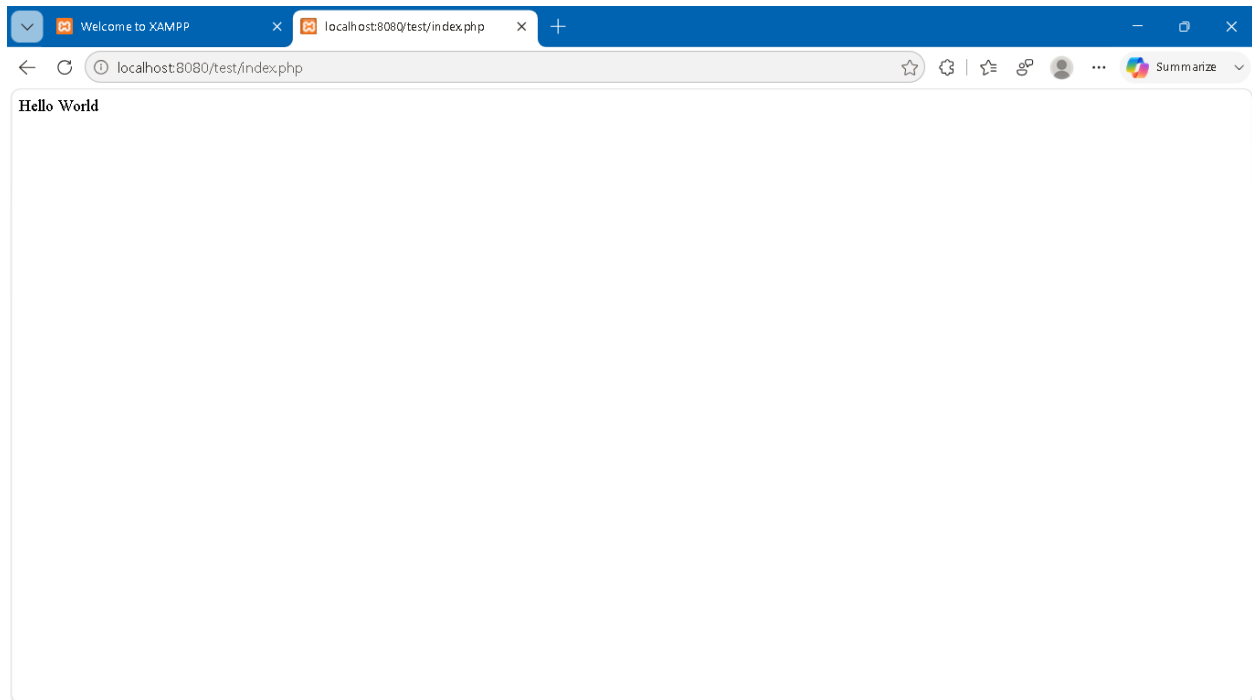
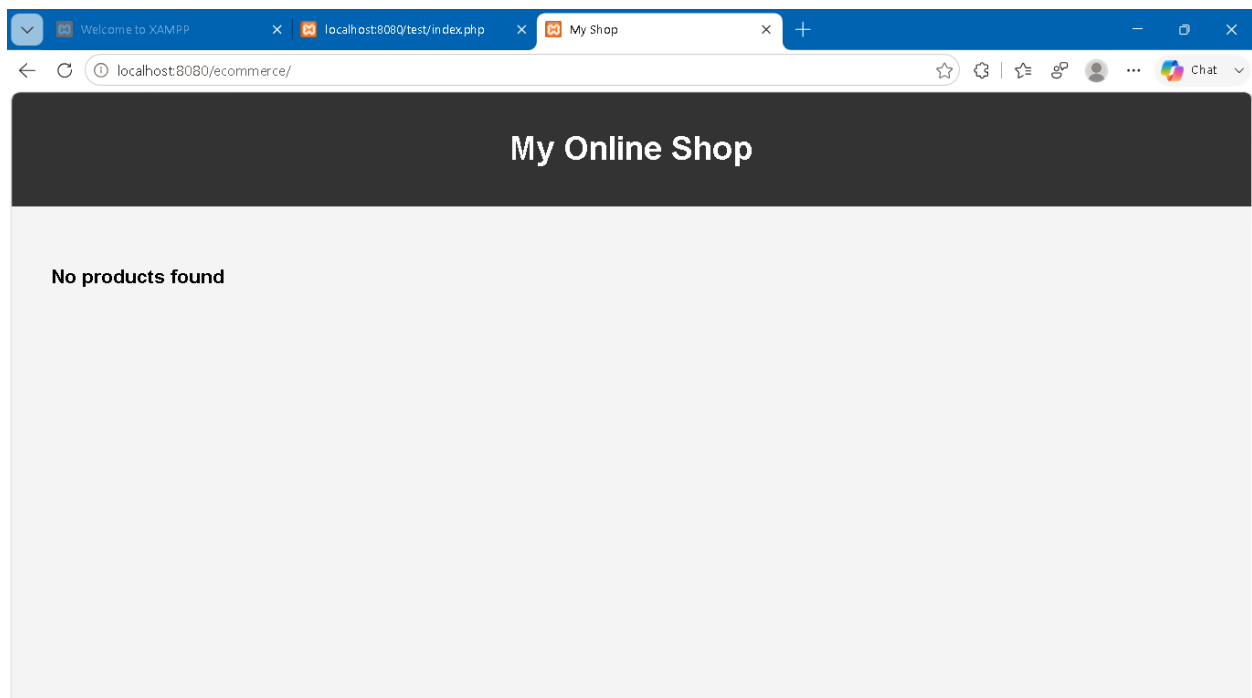


Fig 5: Database Connection Test

Caption:

Successful connection between PHP and MySQL using mysqli_connect().



Student Reflection

During this week, I installed XAMPP and configured the local development environment. Apache and MySQL services were tested successfully. A localhost test confirmed that the web server was functioning properly. I also created and executed a simple PHP page and established a successful connection to MySQL. The screenshots demonstrate successful setup of the development environment required for the e-commerce project.

WEEK 2: WIREFRAMES AND GUI DESIGN

Title:

User Interface Planning and System Design

Required Evidence

Fig 1: Homepage Wireframe

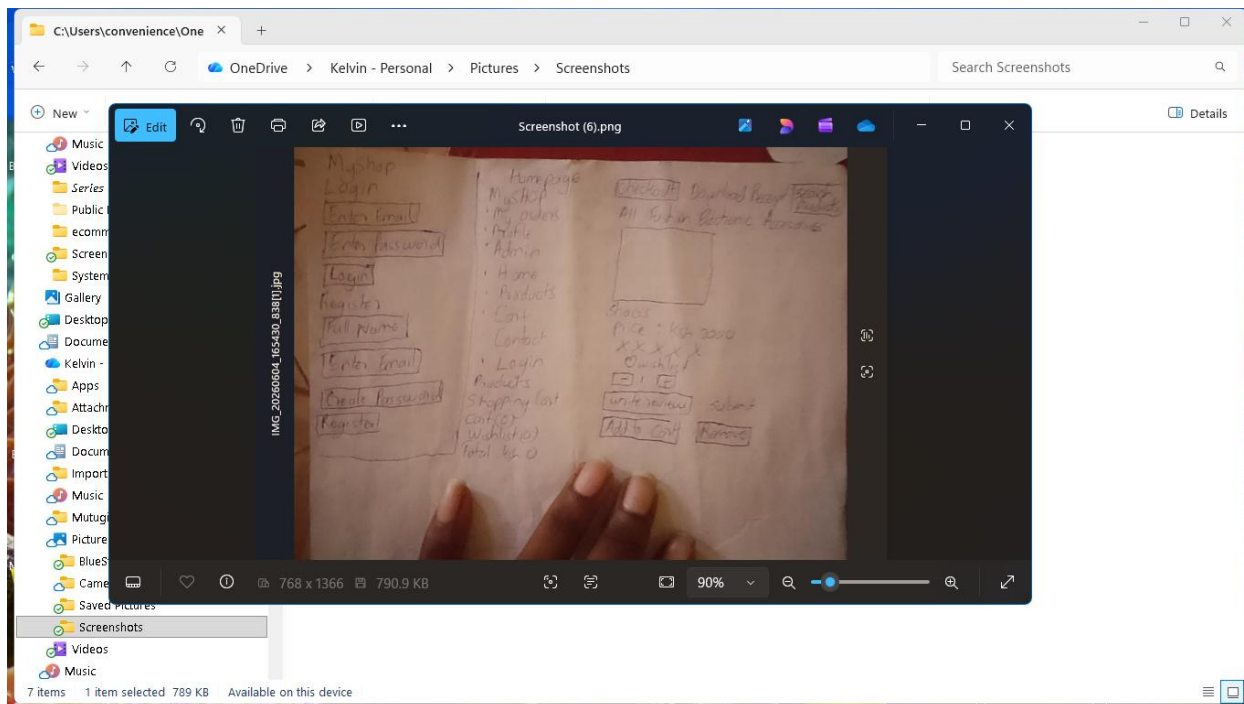


Fig 2: Admin Dashboard Layout Design

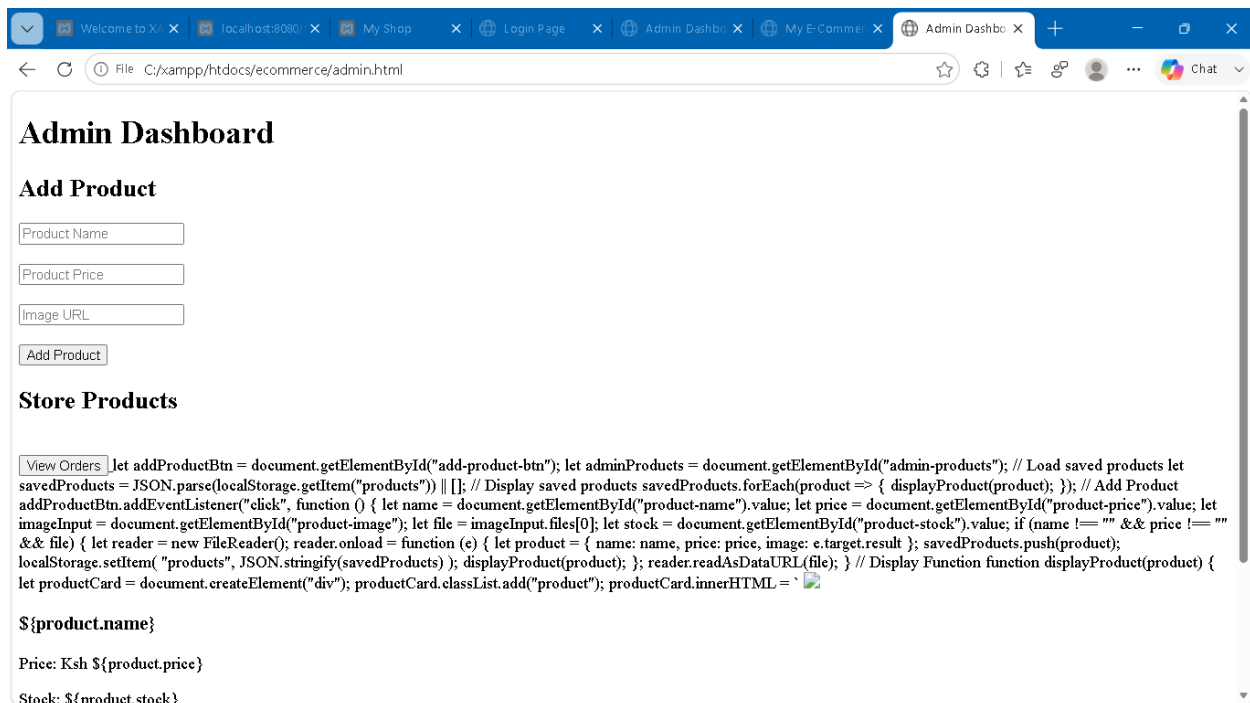


Fig 3: Mobile View Mockup

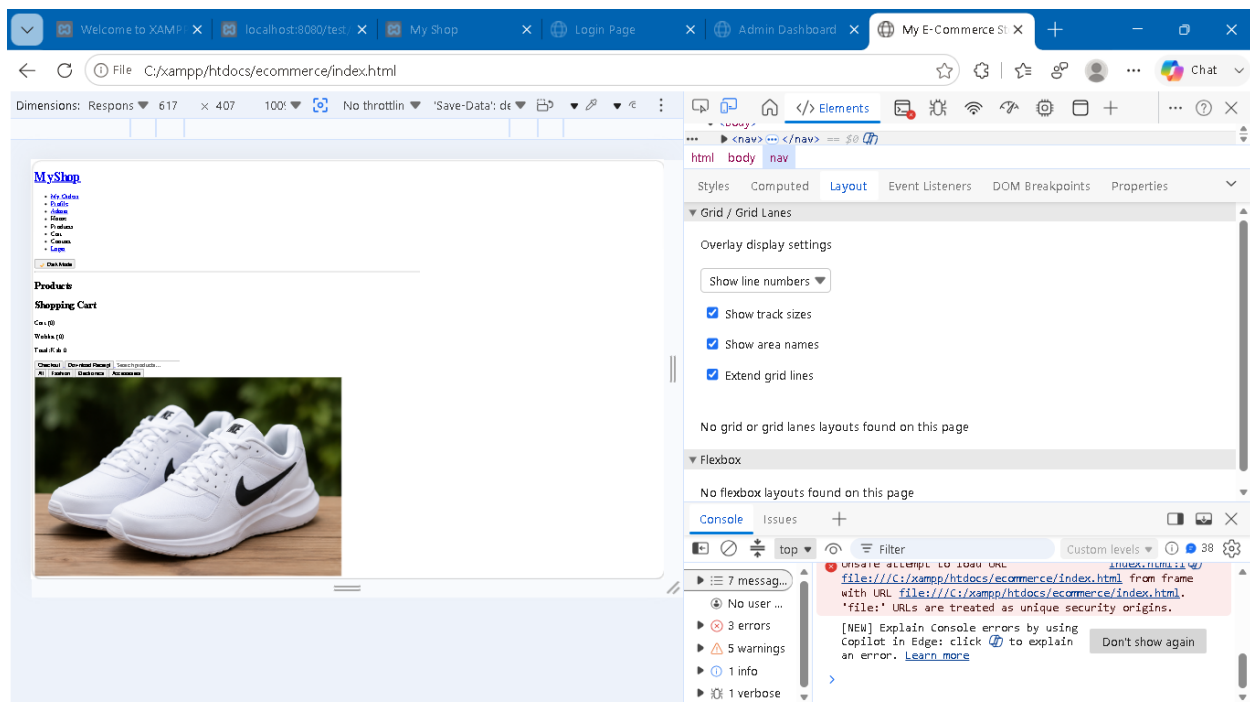


Fig 4: Color Theme and Navigation Structure

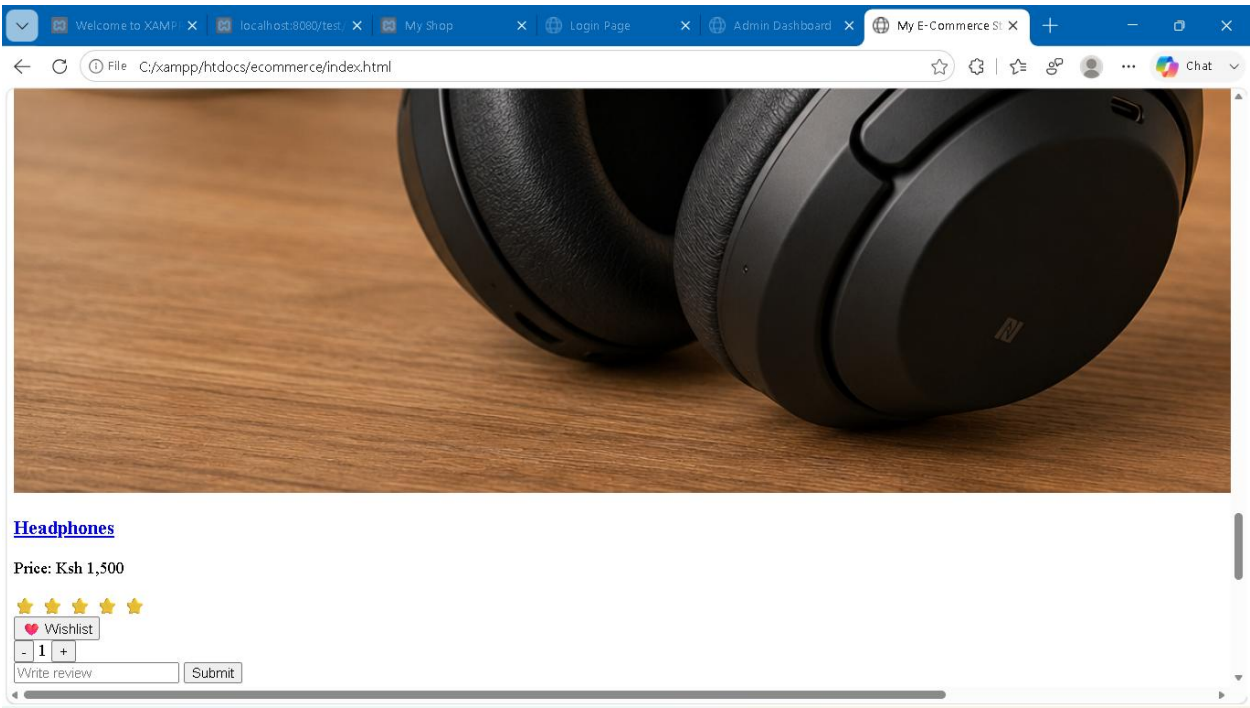
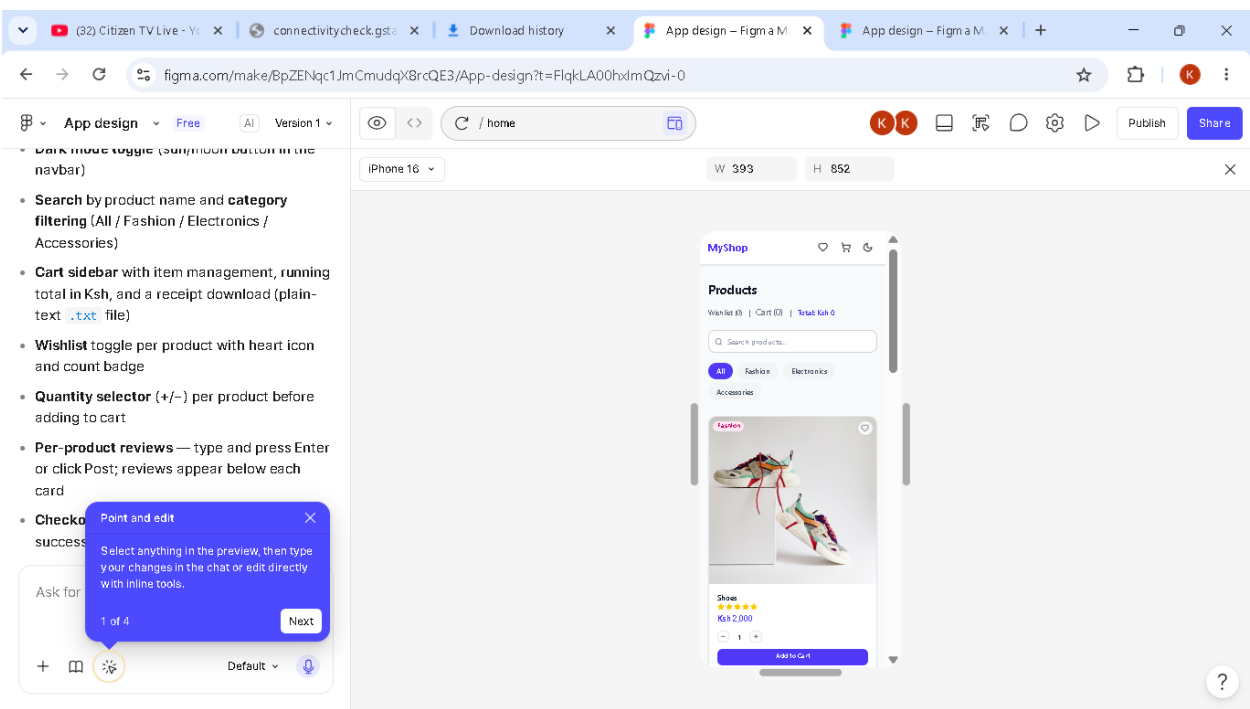


Fig 5: GUI Prototype in Figma/Canva/Draw.io



Student Reflection

The user interface was designed to provide a simple and user-friendly shopping experience. The homepage layout focuses on product visibility while the admin dashboard simplifies product management. Mobile responsiveness was considered to ensure accessibility across different devices. Navigation was planned to minimize user clicks and improve usability.

WEEK 3: JAVASCRIPT AND PHP BASICS

Title:

Frontend Interaction and Backend Foundations

Required Evidence

Fig 1: JavaScript Form Validation

The screenshot shows a web browser window with two tabs: 'bon-livingston/BIT3208-Advance' and 'eCommerce Registration'. The address bar displays 'localhost/ecommerce_smart/user/register.html'. The page content is a 'Create Account' form with three input fields: 'Username:', 'Email Address:', and 'Password:'. Each field has a red border indicating a validation error. Below the 'Username' field is the message 'Username is required.' Below the 'Email Address' field is the message 'Please enter a valid email address.' Below the 'Password' field is the message 'Password must be at least 6 characters long.' At the bottom of the form is a 'Register' button. The browser's taskbar at the bottom shows various application icons and the system clock indicating 7:29 PM on 6/11/2026.

Create Account

Username:

Username is required.

Email Address:

Please enter a valid email address.

Password:

Password must be at least 6 characters long.

Register

Fig 2: Password Strength Checker or Input Validation

The screenshot shows a web browser window with the address bar displaying `localhost/eCommerce_smart/user/register.html`. The page title is "Create Account". The form contains three input fields:

- Username:** The input field contains `bon_livingston`. Below it, a message reads "Username is required."
- Email Address:** The input field contains `bonlivingstone2@gmail.com`. Below it, a message reads "Please enter a valid email address."
- Password:** The input field contains six dots. Below it, a message reads "Password must be at least 6 characters long." and "Strength: Strong (Excellent)".

At the bottom of the form is a "Register" button. The browser's taskbar at the bottom shows the time as 7:31 PM on 6/11/2026.

Fig 3: PHP Syntax Practice

The screenshot shows a code editor window with the file `process_register.php` open. The code is as follows:

```

1  <?php
2  // Call the database structure script
3  require_once '../config/db_connect.php';
4
5  // Verify request context method
6  if ($_SERVER["REQUEST_METHOD"] == "POST") {
7
8      // Capture and clean data inputs dynamically
9      $username = trim($_POST['username']);
10     $email = trim($_POST['email']);
11     $password = trim($_POST['password']);
12
13     // Server-side baseline check
14     if (empty($username) || empty($email) || empty($password)) {
15         die("Error: All system form fields must be populated.");
16     }
17
18     // Input Sanitization for system safety
19     $username = htmlspecialchars(mysqli_real_escape_string($conn, $username));
20     $email = filter_var($email, FILTER_SANITIZE_EMAIL);
21
22     // Password Hashing via native PHP encryption algorithms
23     $hashed_password = password_hash($password, PASSWORD_BCRYPT);
24
25     // Formulate SQL Query to insert into the users table
26     $sql = "INSERT INTO accounts (username, email, password) VALUES ('$username', '$email', '$hashed_password')";
27
28     // Execute dynamic storage operations using procedural connection
29     if (mysqli_query($conn, $sql) === TRUE) {
30         echo "<div style='padding:20px; background:#d4edda; color:#155724; border: 1px solid #c3e6cb;'>
31         echo "<h3>Success!</h3> Account created dynamically for user: <strong>".$username."</strong>
32         echo "<button type='button' href='register.html'>Return to Registration</button>";

```

The editor's Explorer panel on the left shows a project structure for `ECOMMERCE_SMART` with folders like `admin`, `assets`, `css`, `images`, `js`, `cart`, `config`, `includes`, `products`, `user`, and `test_db`. The `process_register.php` file is highlighted in the `user` folder. The status bar at the bottom indicates the cursor is at line 26, column 33, with 4 spaces, UTF-8 encoding, CRLF line endings, and PHP syntax.

Fig 4: Database Connection Script

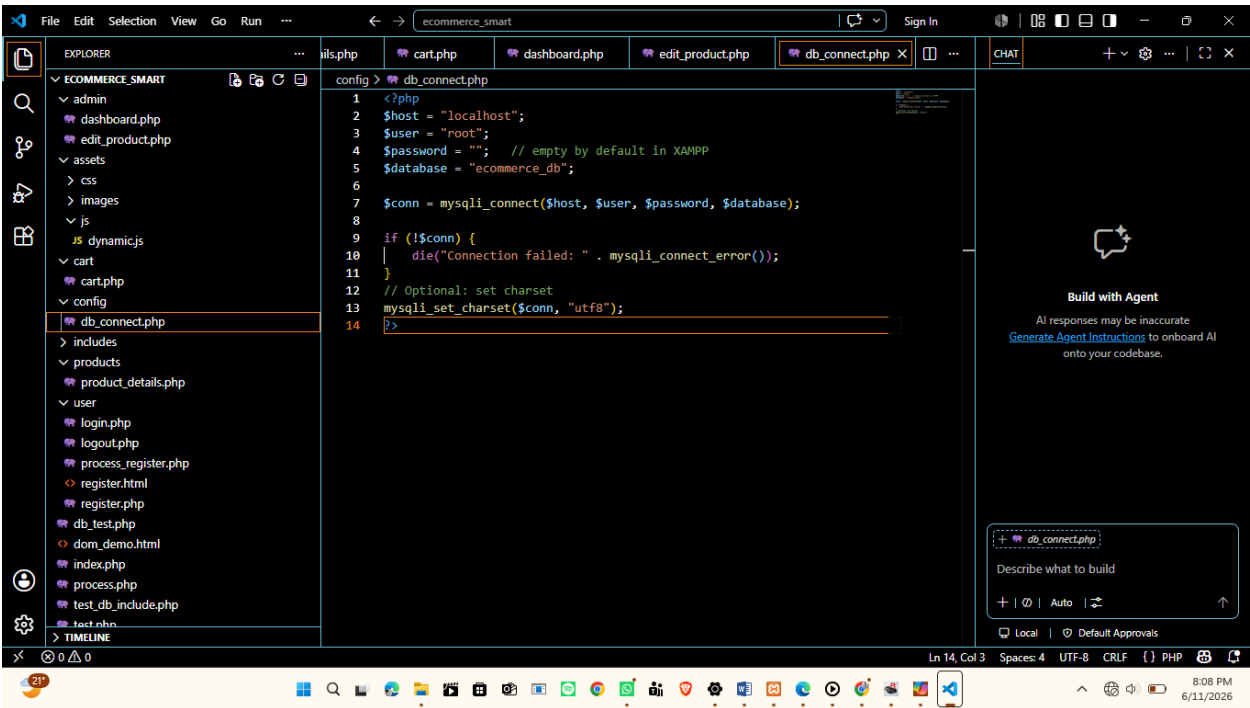
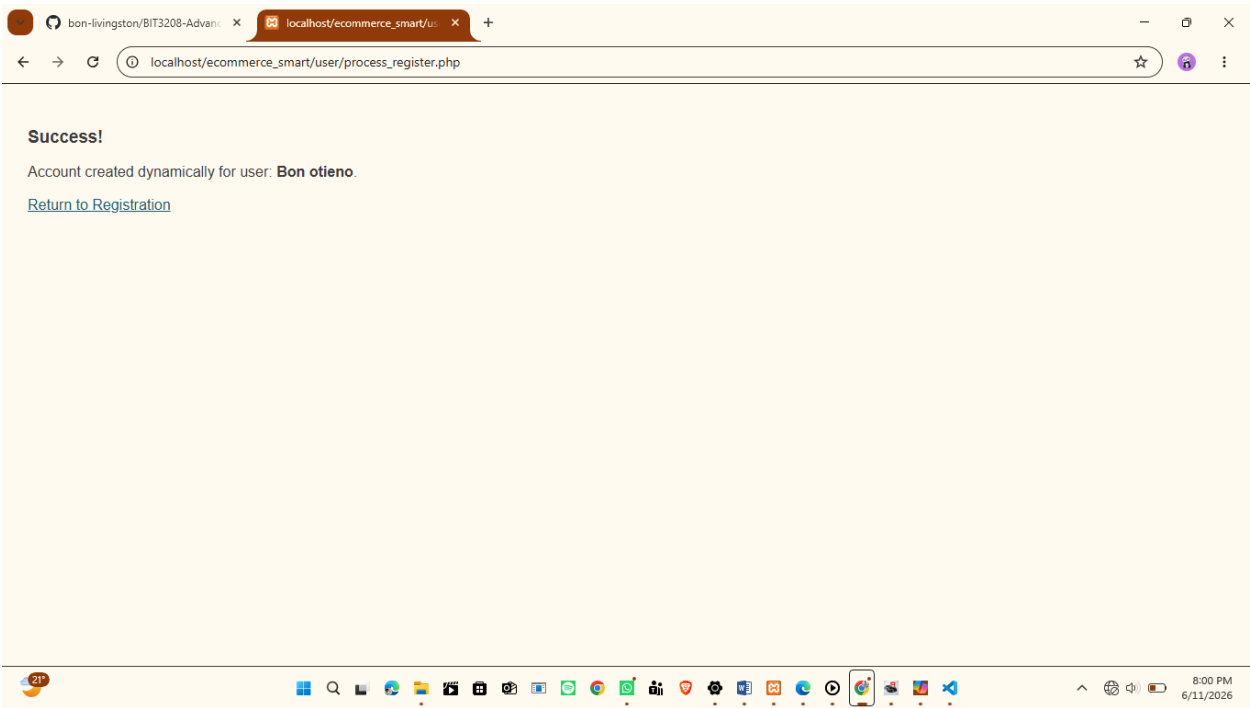


Fig 5: Dynamic User Input Handling



Student Reflection

This week focused on learning frontend and backend interactions. JavaScript was used to validate form inputs before submission. PHP was used to process user requests and communicate with the database. These activities improved my understanding of how user input is handled securely and efficiently in web applications.

=====

WEEK 4: DATABASE DESIGN AND INTEGRATION

Title:

Database Creation and Backend Integration

Required Evidence

Fig 1: Creation of e commerce_db Database

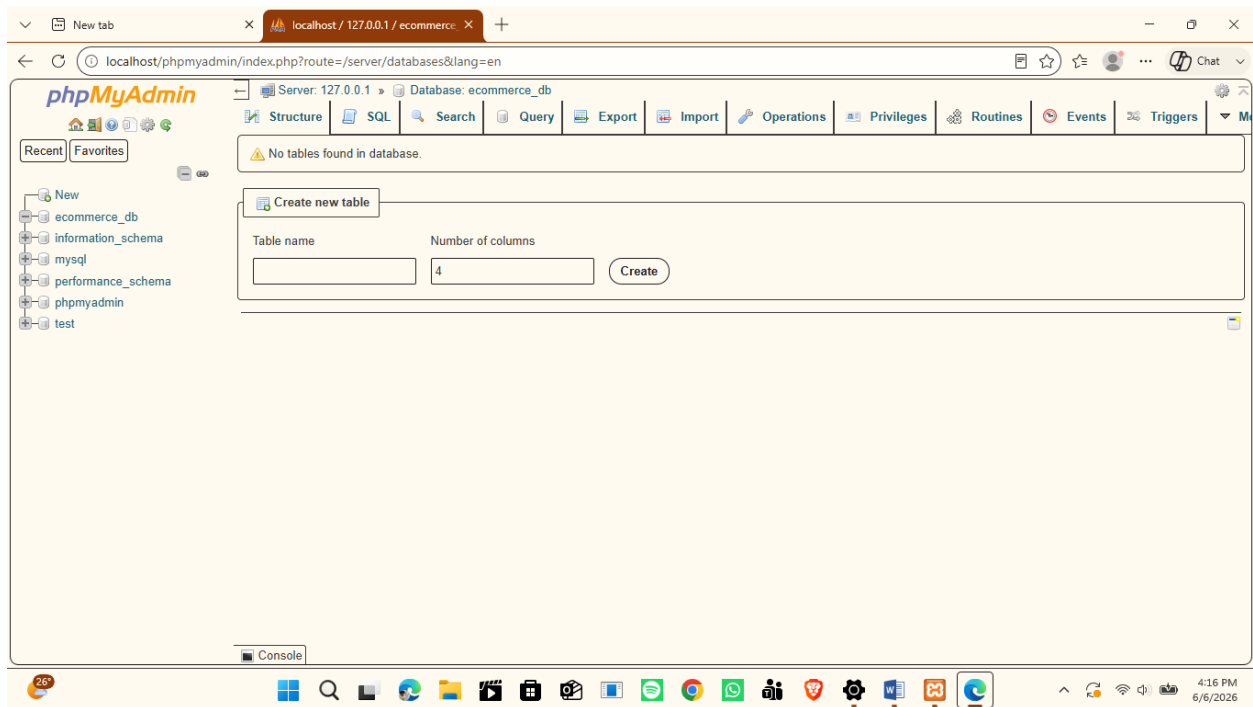


Fig 2: Products Table Structure in phpMyAdmin

The screenshot displays the phpMyAdmin interface for the 'ecommerce_db' database, specifically the 'Table structure' view for the 'productItems' table. The table structure is as follows:

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	ProductID	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	ProductName	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
3	Category	varchar(50)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	Price	decimal(10,2)			No	None			Change Drop More
5	Quantity	int(11)			Yes	0			Change Drop More
6	CreatedAt	timestamp			No	current_timestamp()			Change Drop More

Below the table structure, there are options to 'Check all', 'With selected', 'Browse', 'Change', 'Drop', 'Primary', 'Unique', 'Index', 'Spatial', 'Fulltext', and 'Add to central columns'. There is also a 'Remove from central columns' option.

The 'Indexes' section shows the primary key for 'ProductID' with the following details:

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	ProductID	0	A	No	

At the bottom, there is a 'Partitions' section with a 'Console' tab.

Fig 3: SQL Commands Used for Table Creation

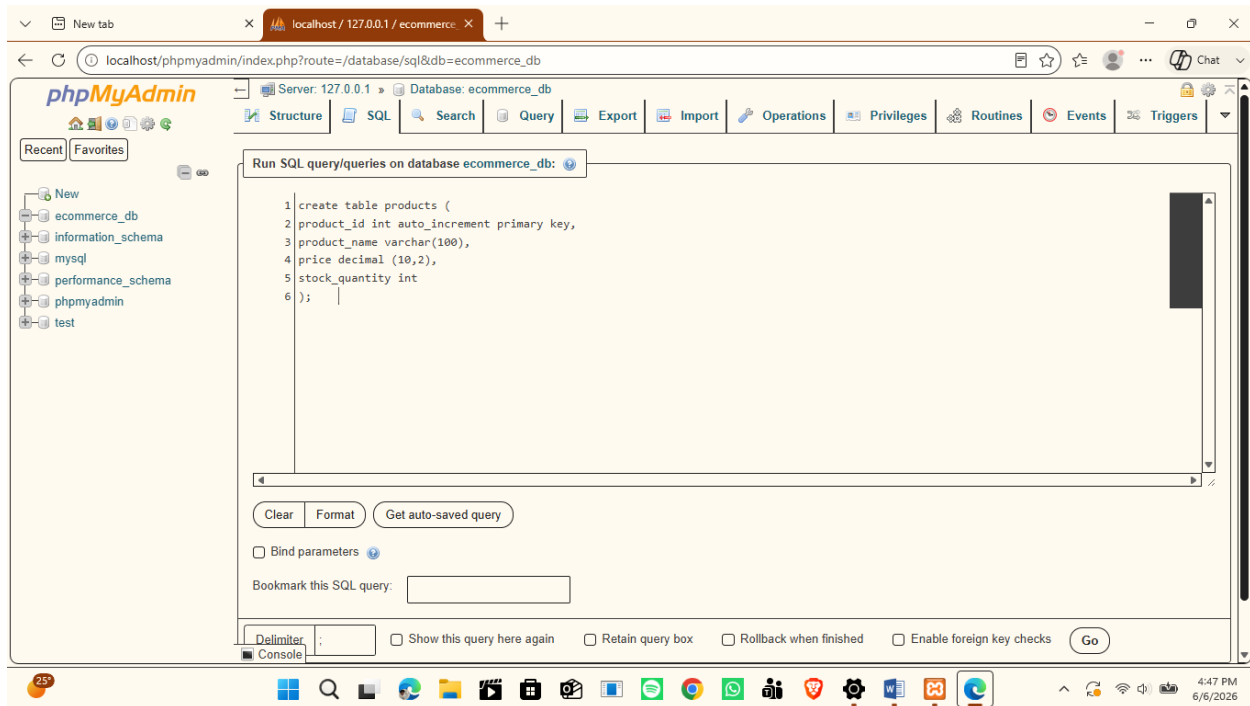


Fig 4: Product Insertion Test

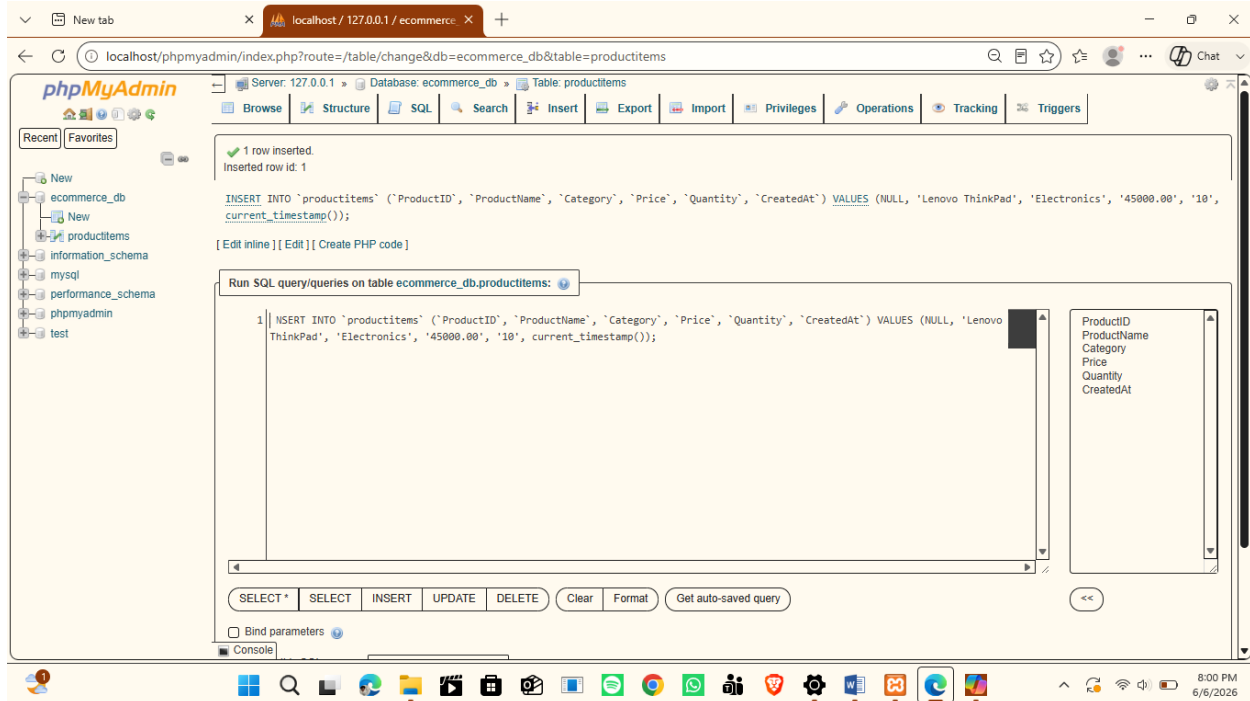
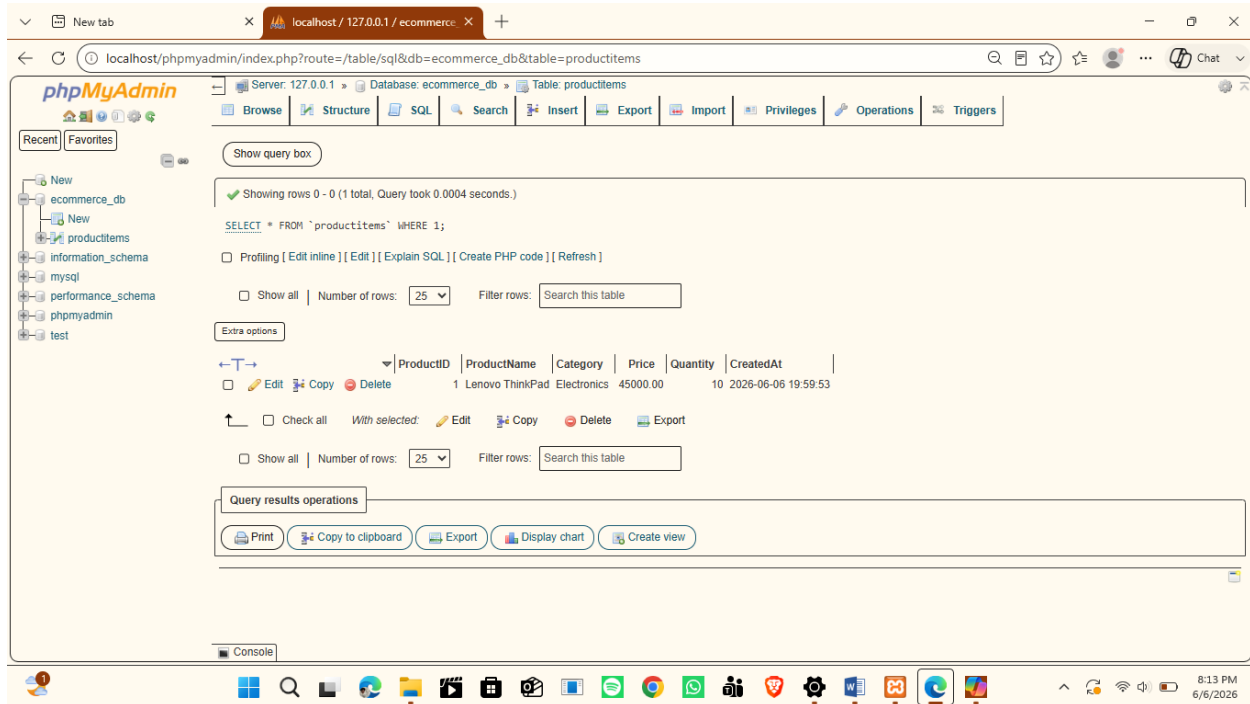


Fig 5: Viewing Records in phpMyAdmin



Student Reflection

The ecommerce_db database was created and connected successfully to the website. A products table was designed to store product information such as product name, price and image. SQL commands were used to create the database structure and verify successful insertion of records. This week strengthened my database management skills.

=====

WEEK 5: PRODUCT MANAGEMENT MODULE

Title:

Development of Product Management Functionality

Required Evidence

Fig 1: Admin Product Entry Form

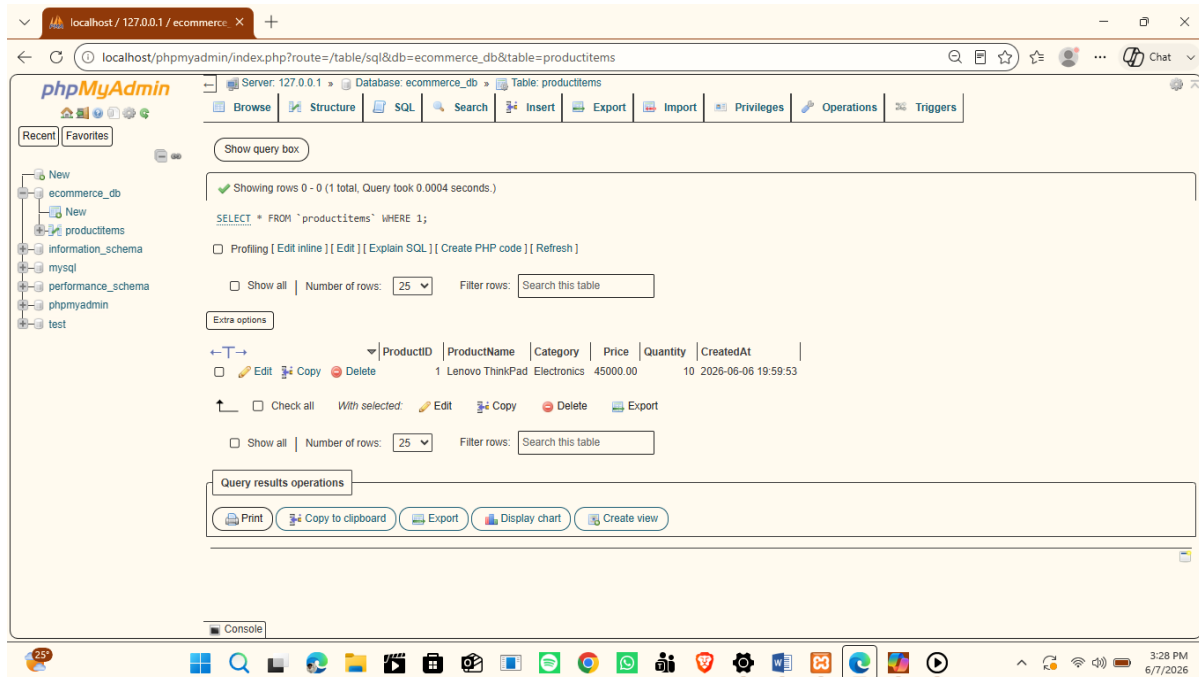


Fig 2: Product Submission Process

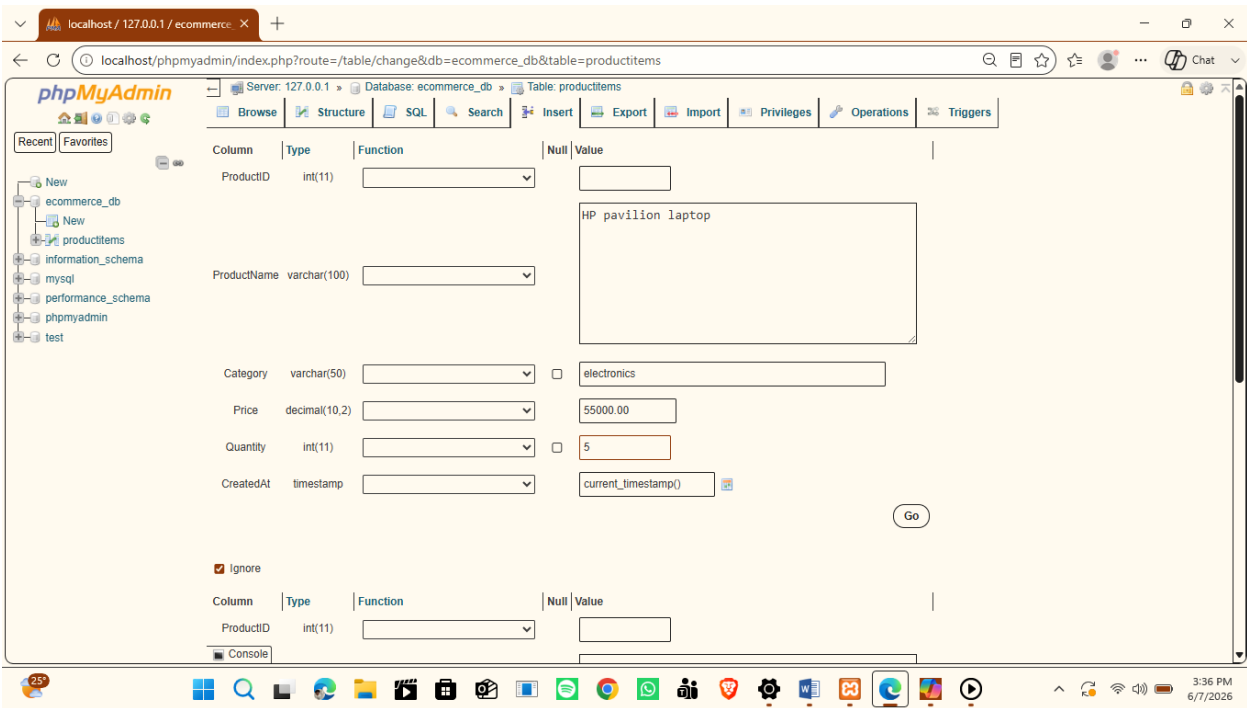


Fig 3: Product Successfully Added to Database

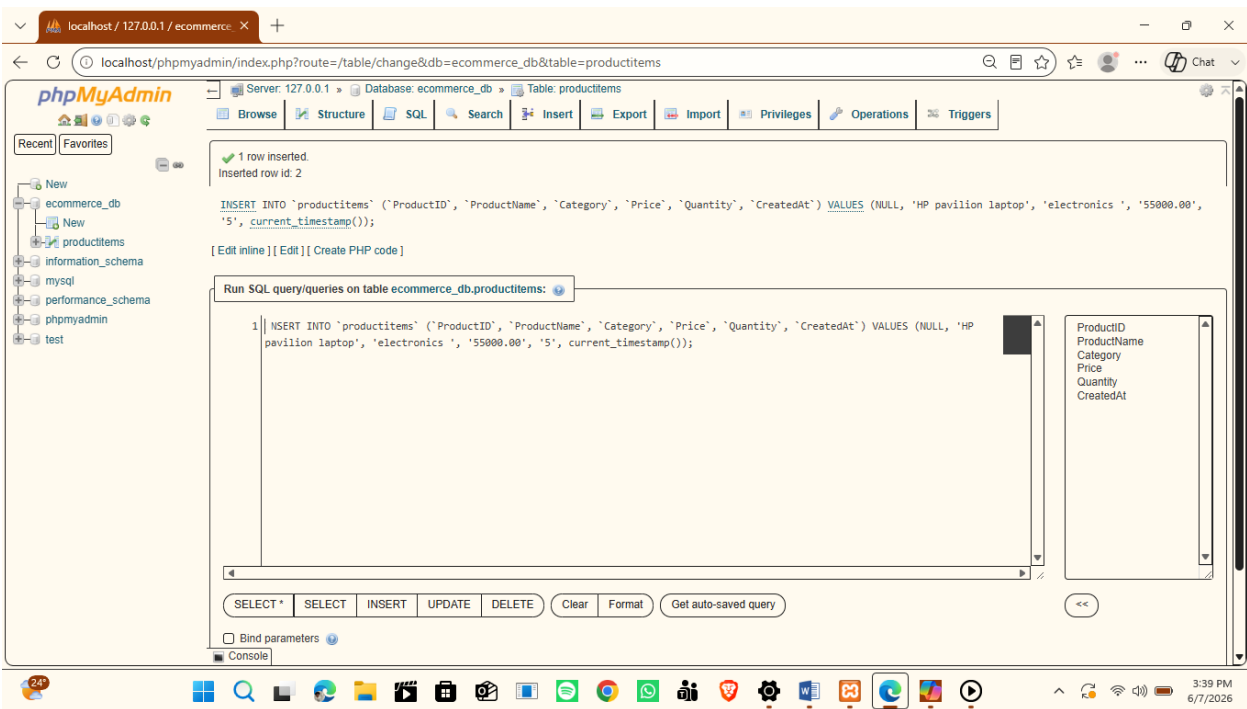


Fig 4: Product Display Page (index.php)

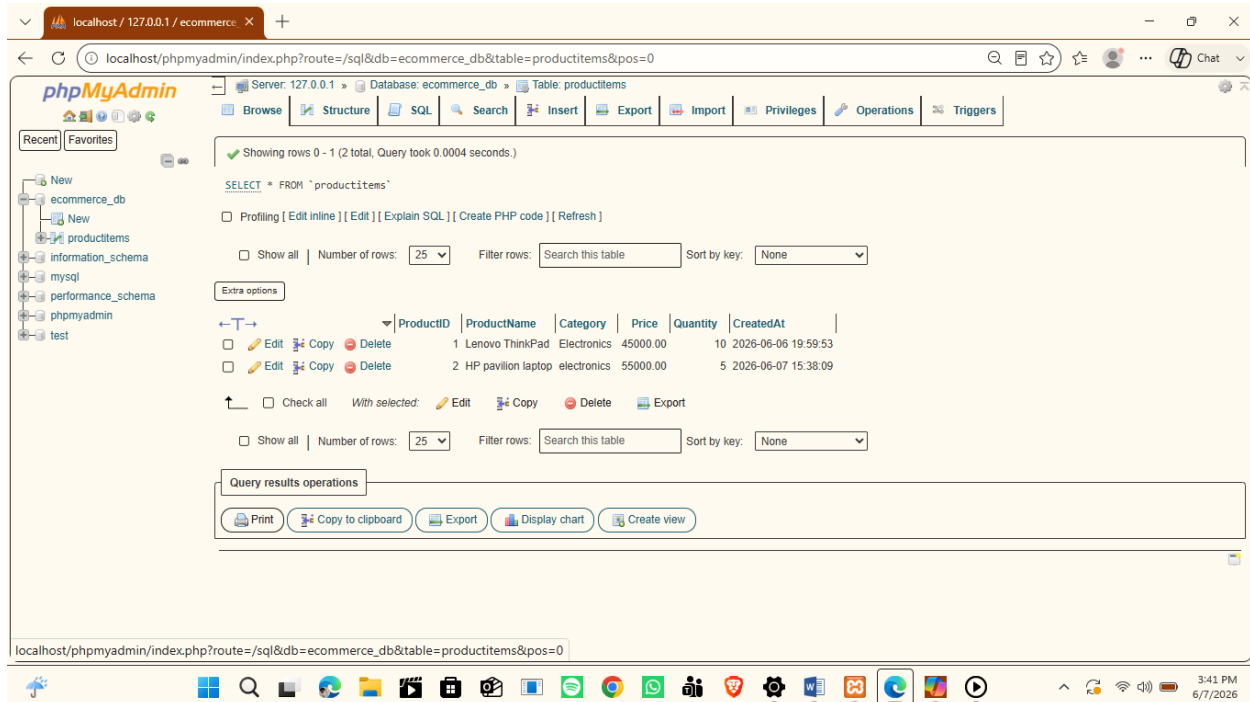
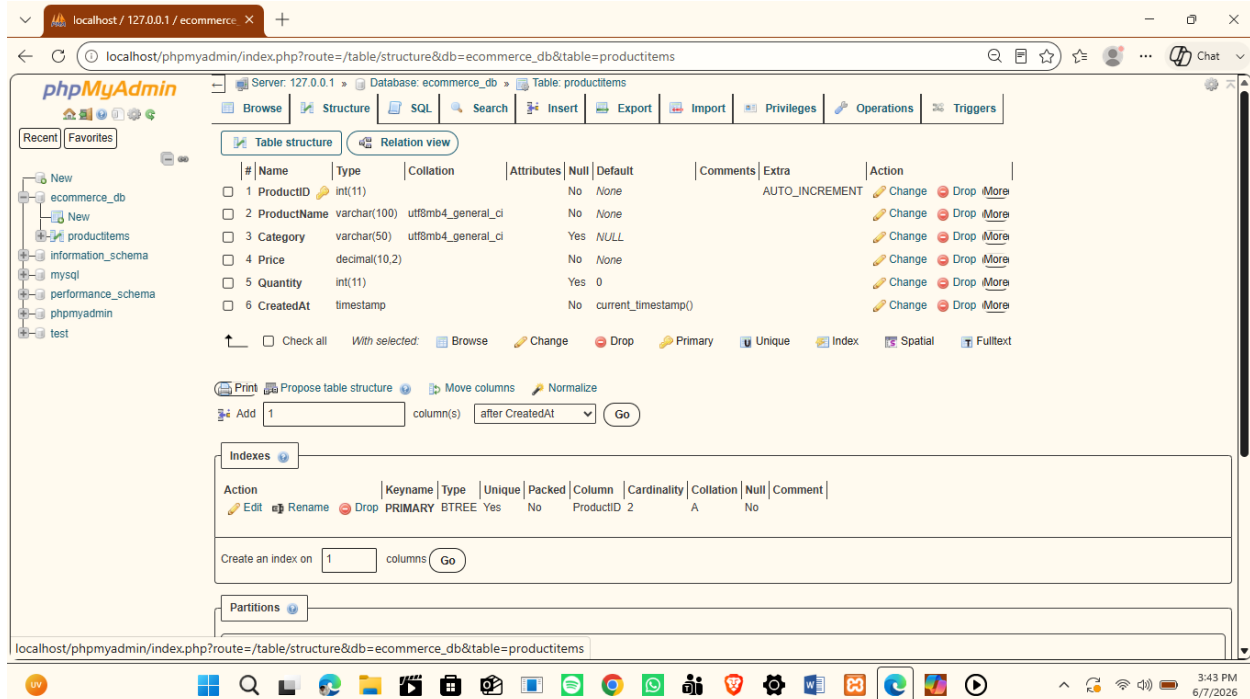


Fig 5: Testing and Debugging Product Workflow



Student Reflection

This week involved developing the product management module. The admin page was created to allow product entry into the database. PHP scripts were used to insert product information into MySQL and retrieve records for display on the homepage. Testing and debugging were carried out to ensure products were stored and displayed correctly.